# Introduction to Quantum Hardware

QML

## Contents


1 Overview 3

2 The Quantum Hardware Landscape 3

2.1 Overview of Platforms . . . 3

3 Superconducting Qubits — Deep Dive 4

3.1 The Josephson Junction . . . 4

3.2 The Transmon Qubit . . . 4

3.3 Coupling Mechanisms . . . 5

4 Qubit Connectivity and Topology 5

4.1 IBM's Heavy-Hex Lattice . . . 5

4.2 Google's Square Grid . . . 6

4.3 Impact on QML Circuits . . . 6

5 Gate Fidelities 7

5.1 Typical Fidelity Numbers (2024) . . . 7

5.2 How Fidelity Limits Circuit Depth . . . 7

6 Coherence Times 8

6.1 T1 — Energy Relaxation . . . 8

6.2 T2 — Dephasing . . . 8

6.3 What This Means for Circuit Depth . . . 9

7 Calibration and Tuneup 9

7.1 How Gates Are Calibrated . . . 9

7.2 Why Fidelities Drift . . . 9

8 IBM Quantum Systems 10

8.1 Eagle Processor (127 Qubits) . . . 10

8.2 Heron Processor (133 Qubits) . . . 10

8.3 IBM Roadmap . . . 10

9 Quantum Volume and CLOPS 11
9.1 Quantum Volume . . . 11
9.2 CLOPS — Circuit Layer Operations Per Second . . . 11

10 Accessing Hardware: Qiskit Runtime Primitives 12
10.1 Estimator . . . 12
10.2 Sampler . . . 12
10.3 Error Mitigation . . . 12

11 Why Hardware Matters for QML 13
11.1 Feature Map Depth Constraints . . . 13
11.2 Noise Budget Analysis . . . 13
11.3 Circuit Depth Limits Per Platform . . . 13

12 Trapped Ions 14
12.1 The Molmer–Sorensen (MS) Gate . . . 14
12.2 All-to-All Connectivity . . . 14
12.3 Disadvantages . . . 14

13 Photonic Quantum Computing 14
13.1 Continuous Variable Approach . . . 15
13.2 Gaussian Boson Sampling . . . 15
13.3 Photonic QML . . . 15

14 Neutral Atoms 15
14.1 Rydberg Interaction . . . 15
14.2 Reconfigurable Arrays . . . 16
14.3 Native Gates . . . 16

15 Platform Comparison for QML 16

16 Practical Considerations 17
16.1 Choosing Qubits on a Backend . . . 17
16.2 Shot Budget . . . 17
16.3 Error Mitigation Overhead . . . 18

17 Summary and Outlook 18

# 1 Overview

up until now weve been treating quantum computers as perfect mathematical objects — u write a circuit, it runs exactly as specified, and u get the right answer. in reality, thats very far from the truth. real quantum hardware is noisy, limited in connectivity, has finite coherence times, and each gate introduces a small amount of error that accumulates as ur circuit gets deeper.

this lecture is about understanding the actual physical machines that run ur QML circuits. why does this matter? bcs the gap between "ideal quantum computer" and "real quantum device" determines whether ur fancy variational ansatz actually works or just produces garbage. if u design a 50-layer feature map but the hardware can only faithfully execute 10 layers before noise dominates, ur model is useless.

well cover the major hardware platforms — superconducting qubits (IBM, Google), trapped ions (IonQ, Quantinuum), photonic systems (Xanadu), and neutral atoms (QuEra) — with a deep dive into superconducting systems since thats wt most of u will use through IBM Quantum. well talk about gate fidelities, coherence times, qubit connectivity, and how all of this constrains the QML circuits u can actually run.

the key takeaway: hardware awareness isnt optional for QML. its the difference between circuits that work and circuits that dont.

# 2 The Quantum Hardware Landscape

there r four major approaches to building quantum computers that r actively being pursued by companies and research labs. each has different strengths and weaknesses, and these differences directly affect wt kinds of QML algorithms u can run.

## 2.1 Overview of Platforms

| Property | Superconducting | Trapped Ions | Photonic | Neutral Atoms |
|---|---|---|---|---|
| qubit type | transmon | atomic ions | photon modes | neutral atoms |
| connectivity | nearest-neighbor | all-to-all | reconfigurable | reconfigurable |
| gate speed | ∼10–100 ns | ∼1–100 $\mu$s | ∼1 ns | ∼0.1–1 $\mu$s |
| T1/T2 | ∼100–300 $\mu$s | ∼1–100 s | N/A (photon loss) | ∼1–10 s |
| 2Q gate fidelity | 99–99.9% | 99.5–99.9% | ∼95–99% | 99–99.5% |
| scale (2024) | 100–1000+ Q | 20–56 Q | 100+ modes | 48–256 Q |
| key players | IBM, Google | IonQ, Quantinuum | Xanadu, PsiQuantum | QuEra, Pasqal |
| QML suitability | high (access) | high (connectivity) | niche (CV) | emerging |

Key Point. for QML, the two most important hardware properties r: (1) gate fidelity — determines how deep ur circuits can be before noise dominates, and (2) connectivity — determines how many SWAP gates u need to implement ur desired circuit, which adds depth and noise.

# 3 Superconducting Qubits — Deep Dive

superconducting qubits r the most widely deployed platform and the one most of u will interact with through IBM Quantum. lets understand how they actually work.

## 3.1 The Josephson Junction

the heart of a superconducting qubit is the josephson junction — two superconducting electrodes separated by a thin insulating barrier (typically aluminum oxide, ∼1–2 nm thick).

Definition 3.1 (Josephson Junction). a josephson junction is a nonlinear, lossless circuit element with the current-phase relation:

$$I = I_c \sin(\varphi)$$

where $I_c$ is the critical current and $\varphi$ is the superconducting phase difference across the junction. the junction stores energy:

$$U(\varphi) = -E_J \cos(\varphi)$$

where $E_J = \frac{\hbar I_c}{2e}$ is the josephson energy (with $\hbar$ the reduced planck constant).

why does this matter? bcs the nonlinearity of $\cos(\varphi)$ is wt gives us an anharmonic oscillator — and thats wt lets us isolate two energy levels to use as a qubit.

## 3.2 The Transmon Qubit

a regular LC oscillator has equally spaced energy levels — u cant address just two of them bcs a pulse that excites $|0\rangle \to |1\rangle$ would also excite $|1\rangle \to |2\rangle$. the transmon fixes this by replacing the linear inductor with a josephson junction.

the transmon hamiltonian is:

$$\hat{H} = 4E_C(\hat{n} - n_g)^2 - E_J \cos(\hat{\varphi})$$

where $E_C = \frac{e^2}{2C_\Sigma}$ is the charging energy, $\hat{n}$ is the number of cooper pairs, $n_g$ is the offset charge, and $C_\Sigma$ is the total capacitance.

the key design parameter is the ratio $E_J/E_C$. for a transmon, we want $E_J/E_C \gg 1$ (typically 50–100). this makes the qubit insensitive to charge noise (which is great for coherence) at the cost of reduced anharmonicity.

Definition 3.2 (Anharmonicity). the anharmonicity of a transmon is:

$$\alpha = E_{12} - E_{01} = (E_2 - E_1) - (E_1 - E_0) \approx -E_C$$

for a typical transmon, $\alpha/2\pi \approx -300$ MHz, while the qubit frequency is $\omega_{01}/2\pi \approx 5$ GHz. this means the $|0\rangle \to |1\rangle$ transition is ∼300 MHz away from the $|1\rangle \to |2\rangle$ transition — enough to selectively address the qubit subspace.

Intuition. think of a transmon as a pendulum. a harmonic oscillator is like a spring — all oscillation periods r the same regardless of amplitude. a pendulum has the property that larger swings take longer. this “anharmonicity” is wt lets u distinguish between different energy levels and treat the bottom two as a qubit.

the energy levels of the transmon r approximately:

$$E_m \approx -E_J + \sqrt{8E_J E_C}\left(m + \frac{1}{2}\right) - \frac{E_C}{12}(6m^2 + 6m + 3)$$

so the qubit frequency is:

$$\omega_{01} = \frac{E_1 - E_0}{\hbar} \approx \frac{1}{\hbar}\left(\sqrt{8E_J E_C} - E_C\right)$$

### 3.3 Coupling Mechanisms

individual qubits r useless without the ability to make them interact. there r two main coupling strategies in superconducting systems.

capacitive coupling: the simplest approach — just put a capacitor between two transmons. this gives a fixed coupling:

$$\hat{H}_{\text{int}} = g(\hat{a}_1^\dagger \hat{a}_2 + \hat{a}_1 \hat{a}_2^\dagger)$$

where $g$ is the coupling strength (typically $g/2\pi \approx 1$–$10$ MHz) and $\hat{a}_i$, $\hat{a}_i^\dagger$ r the annihilation/creation operators for qubit $i$. the problem with fixed coupling is that qubits always interact, leading to unwanted ZZ crosstalk.

resonator-mediated coupling: a more sophisticated approach where qubits couple through a shared microwave resonator (bus resonator). in the dispersive regime (where qubit-resonator detuning $\Delta$ is much larger than the coupling $g$), the effective qubit-qubit interaction is:

$$g_{\text{eff}} = \frac{g_1 g_2}{\Delta}$$

this is how IBM connects qubits in their heavy-hex lattice — each pair of connected qubits shares a coupling resonator.

tunable couplers: modern designs (used in Google's Sycamore and IBM's newer processors) include a tunable coupler between qubits that can turn the effective coupling on and off:

$$g_{\text{eff}}(\Phi) = g_{\text{direct}} + \frac{g_{1c} g_{2c}}{\Delta_c(\Phi)}$$

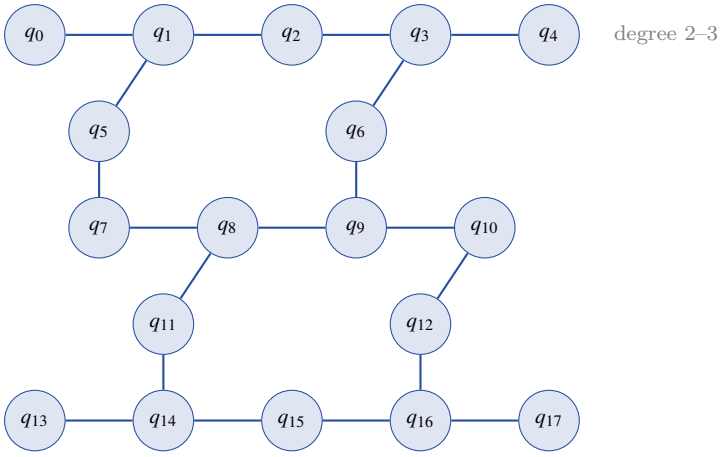
where $\Phi$ is the external flux threading the coupler. by tuning $\Phi$, u can set $g_{\text{eff}} = 0$ (qubits isolated) or $g_{\text{eff}} \neq 0$ (qubits interacting). this dramatically reduces crosstalk errors.

## 4 Qubit Connectivity and Topology

the physical layout of qubits and which pairs can directly interact is called the connectivity or topology. this is arguably the most important hardware constraint for QML bcs it determines how many SWAP operations u need.

### 4.1 IBM's Heavy-Hex Lattice

IBM uses a heavy-hex topology for their eagle (127 qubit) and heron (133 qubit) processors. here is a schematic:




IBM Heavy-Hex Lattice (fragment)

Key Point. in a heavy-hex lattice, every qubit has at most degree 3 (connects to at most 3 neighbors). this means u need a lot of SWAP gates to implement circuits that require interactions between distant qubits. for a QML circuit with all-to-all entanglement on $n$ qubits, u might need $O(n)$ SWAP gates per entangling layer.

why heavy-hex? the low connectivity actually helps with reducing crosstalk errors. when qubits have fewer neighbors, theres less unwanted ZZ interaction, which makes it easier to achieve high gate fidelities. its a deliberate engineering tradeoff: worse connectivity but better per-gate quality.

## 4.2 Google's Square Grid

Google's Sycamore processor uses a 2D square grid where each qubit connects to 4 neighbors (except at edges). this gives better connectivity than heavy-hex but potentially more crosstalk.

the diameter of the graph (maximum shortest path between any two qubits) is smaller for a square grid, meaning fewer SWAPs r needed on average. for an $n \times n$ grid, the diameter is $2(n-1)$, while for heavy-hex its somewhat larger due to the sparse bridges.

## 4.3 Impact on QML Circuits

consider a simple QML ansatz that applies CNOT between every pair of qubits in a 4-qubit system. on hardware with all-to-all connectivity (like trapped ions), this requires exactly 6 CNOT gates. on a linear chain, some of these CNOTs must be decomposed using SWAPs:

each SWAP gate costs 3 CNOT gates. so if ur ansatz needs $k$ non-native CNOT connections, and each requires on average $s$ SWAPs, ur actual two-qubit gate count is:

$$N_{2Q} = N_{\text{native}} + 3 \cdot N_{\text{SWAP}} \cdot s$$

Example 4.1. consider a 6-qubit QML circuit with circular entanglement ($q_0$-$q_1$-$q_2$-$q_3$-$q_4$-$q_5$-$q_0$) on a linear chain. 5 of the 6 CNOT gates r between neighbors and need no SWAPs. the $q_5$-$q_0$ gate requires routing through the entire chain, costing approximately 5 SWAP gates = 15 additional CNOT gates. so the total goes from 6 ideal CNOTs to $5+1+15=21$ actual CNOTs.

# 5 Gate Fidelities

gate fidelity measures how close the actual operation is to the ideal one. this is the single most important number for determining whether ur QML circuit will work.

Definition 5.1 (Average Gate Fidelity). for a quantum channel $\mathscr{E}$ that is supposed to implement a unitary $U$, the average gate fidelity is:

$$F_{\text{avg}} = \int d\psi \, \langle\psi| U^\dagger \mathscr{E}(|\psi\rangle\langle\psi|) U |\psi\rangle$$

where the integral is over uniformly distributed pure states.

in practice, fidelities r measured using randomized benchmarking (RB), which gives the error per clifford gate, or more recently interleaved RB for specific gates.

## 5.1 Typical Fidelity Numbers (2024)

| Gate Type | Typical Fidelity | Error Rate |
|---|---|---|
| single-qubit gate (SX, RZ) | 99.9–99.99% | $10^{-3}$–$10^{-4}$ |
| two-qubit gate (ECR, CX) | 99.0–99.9% | $10^{-2}$–$10^{-3}$ |
| measurement (readout) | 98.0–99.5% | $5\times10^{-3}$–$2\times10^{-2}$ |
| state preparation ($\|0\rangle$) | 99.5–99.9% | $10^{-3}$–$5\times10^{-3}$ |

## 5.2 How Fidelity Limits Circuit Depth

heres the key calculation for QML. if each layer of ur circuit has $n_{1Q}$ single-qubit gates and $n_{2Q}$ two-qubit gates, and u have $L$ layers, the total circuit fidelity is approximately:

$$F_{\text{circuit}} \approx F_{1Q}^{L \cdot n_{1Q}} \cdot F_{2Q}^{L \cdot n_{2Q}} \cdot F_{\text{meas}}^{n}$$

where $n$ is the number of qubits measured.

taking logarithms and using $\ln(1-\varepsilon) \approx -\varepsilon$ for small error:

$$\ln F_{\text{circuit}} \approx -L \cdot n_{1Q} \cdot \varepsilon_{1Q} - L \cdot n_{2Q} \cdot \varepsilon_{2Q} - n \cdot \varepsilon_{\text{meas}}$$

Example 5.1 (Noise Budget for a QML Circuit). say u have a 10-qubit QML circuit with $L$ layers, each layer having 10 single-qubit gates ($\varepsilon_{1Q} = 10^{-3}$) and 9 two-qubit gates ($\varepsilon_{2Q} = 5\times10^{-3}$). readout error is $\varepsilon_{\text{meas}} = 10^{-2}$.

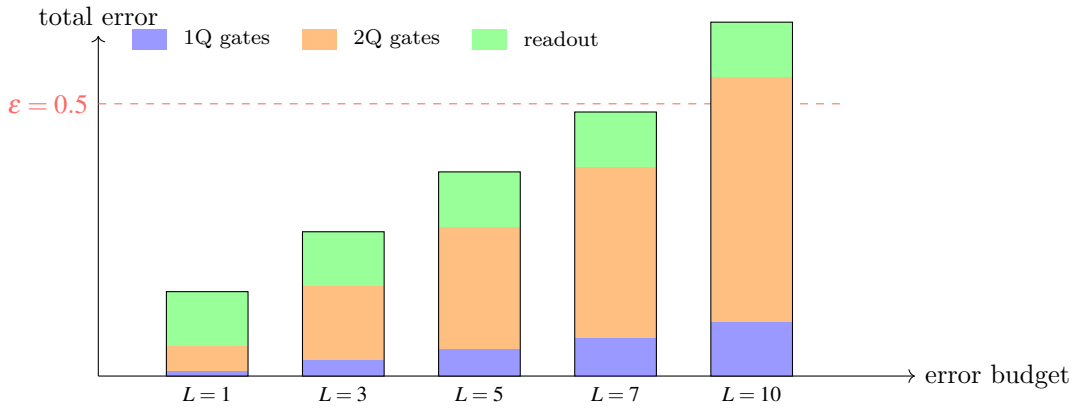
total error:

$$\begin{aligned}\varepsilon_{\text{total}} &\approx L(10\times10^{-3} + 9\times5\times10^{-3}) + 10\times10^{-2} \\ &= L(0.01 + 0.045) + 0.1 = 0.055L + 0.1\end{aligned}$$

for the circuit to have more signal than noise, we roughly need $\varepsilon_{\text{total}} < 0.5$:

$$0.055L + 0.1 < 0.5 \implies L < 7.3$$

so u can run at most $\sim$7 layers. this is a hard constraint on ur QML ansatz depth.




Noise Budget vs. Circuit Depth (10 qubits)

# 6 Coherence Times

coherence times tell u how long a qubit can maintain quantum information before the environment destroys it. there r three key timescales.

## 6.1 T1 — Energy Relaxation

Definition 6.1 ($T_1$ Time). $T_1$ is the timescale for energy relaxation — the qubit spontaneously decays from $|1\rangle$ to $|0\rangle$. if u prepare $|1\rangle$ and wait time $t$, the probability of still being in $|1\rangle$ is:

$$P_1(t) = e^{-t/T_1}$$

this is analogous to the $T_1$ relaxation in NMR. for current transmons, $T_1 \approx 100$–$300\,\mu$s.

$T_1$ is caused by coupling to the electromagnetic environment — the qubit radiates energy into modes it shouldnt be coupled to. dielectric loss in the substrate and junction is a major source.

## 6.2 T2 — Dephasing

Definition 6.2 ($T_2$ and $T_2^*$ Times). $T_2$ (measured with a spin echo or Hahn echo sequence) captures dephasing — the loss of phase coherence between $|0\rangle$ and $|1\rangle$. if u prepare $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and wait, the off-diagonal elements of the density matrix decay as:

$$\rho_{01}(t) = \rho_{01}(0)\, e^{-t/T_2}$$

$T_2^*$ (measured with a Ramsey experiment, no echo) includes both intrinsic dephasing and low-frequency noise:

$$\frac{1}{T_2^*} = \frac{1}{2T_1} + \frac{1}{T_\varphi}$$

where $T_\varphi$ is the pure dephasing time. always $T_2^* \leq T_2 \leq 2T_1$.

Intuition. think of $T_1$ as the qubit "forgetting whether its 0 or 1" and $T_2$ as the qubit "forgetting the phase angle between 0 and 1". for QML, $T_2$ is usually more important bcs most quantum algorithms rely heavily on coherent superpositions, and dephasing kills those superpositions.

### 6.3 What This Means for Circuit Depth

the maximum number of gate layers u can execute is bounded by:

$$L_{\max} \approx \frac{T_2}{t_{\text{gate}}}$$

where $t_{\text{gate}}$ is the duration of one layer. for a two-qubit gate taking $\sim$300 ns and $T_2 \approx 200\,\mu$s:

$$L_{\max} \approx \frac{200 \times 10^3 \text{ ns}}{300 \text{ ns}} \approx 667 \text{ layers}$$

but wait — this is a theoretical maximum assuming perfect gates. in practice, gate errors accumulate much faster than decoherence, so the actual limit is usually set by gate fidelity (as we calculated above), not coherence time. on current hardware, gate errors limit u to $\sim$5–20 useful layers, well below the coherence limit.

## 7 Calibration and Tuneup

gates on a quantum computer r not fixed hardware components like logic gates on a classical chip. they r carefully calibrated microwave pulses, and their fidelity drifts over time.

### 7.1 How Gates Are Calibrated

a single-qubit gate on a transmon is implemented by a microwave pulse at the qubit frequency $\omega_{01}$. to calibrate, say, an $X$ gate (which should flip $|0\rangle$ to $|1\rangle$), u need to find:

(1) qubit frequency $\omega_{01}$: found using spectroscopy (sweep microwave frequency, look for absorption)

(2) pulse amplitude: found using Rabi oscillations (sweep amplitude, find the $\pi$-pulse)

(3) DRAG parameter: corrects for leakage to the $|2\rangle$ state (derivative removal by adiabatic gate)

(4) readout parameters: discriminator calibration for telling $|0\rangle$ from $|1\rangle$ in the IQ plane

two-qubit gates r even more involved — for an ECR (echoed cross-resonance) gate, u need to calibrate the cross-resonance drive amplitude, duration, and phase, plus single-qubit corrections.

### 7.2 Why Fidelities Drift

qubit parameters drift due to:

- TLS defects: two-level systems in the substrate that couple to the qubit and shift its frequency on timescales of minutes to hours
- temperature fluctuations: small changes in the cryostat temperature affect qubit frequencies
- cosmic rays: high-energy particles can temporarily heat the chip and degrade all qubits simultaneously (recently observed by Google)

- quasiparticle poisoning: broken cooper pairs that tunnel across the junction and shift the qubit frequency

IBM recalibrates their systems approximately daily, and publishes calibration data (gate errors, $T_1$, $T_2$, readout errors) for each backend.

Key Point. for QML experiments, always check the latest calibration data before submitting jobs. a qubit that was great yesterday might be terrible today. use the `BackendProperties` API in Qiskit to query current error rates and choose the best qubits for ur circuit.

# 8 IBM Quantum Systems

lets look at the specific processors u can access through IBM Quantum.

## 8.1 Eagle Processor (127 Qubits)

the IBM Eagle processor, introduced in 2021, was the first to break 100 qubits. key specs:

- 127 qubits in heavy-hex topology
- median CNOT error: $\sim$ 1–2%
- median $T_1$: $\sim 100$–$200\,\mu$s
- used in the ibm_brisbane, ibm_osaka, etc. backends

## 8.2 Heron Processor (133 Qubits)

the IBM Heron processor (2023–2024) represents a significant upgrade:

- 133 qubits, still heavy-hex
- tunable couplers (big improvement over fixed coupling in Eagle)
- native ECR gate replaced with improved two-qubit gate
- median two-qubit gate error: $\sim 0.3$–$0.5\%$ (roughly 3–5x better than Eagle)
- used in the ibm_fez, ibm_marrakesh, etc. backends

## 8.3 IBM Roadmap

IBMs roadmap includes:

- Flamingo (2025): modular architecture linking multiple chips via quantum interconnects
- Starling/Blue Jay: 200+ qubit processors with further error reduction
- 100,000 qubit target by 2033

for QML, the key milestone is when two-qubit gate errors drop below $\sim 0.1\%$ — at that point, circuits with $\sim$50+ meaningful layers become feasible, opening up much deeper ansatze.

# 9 Quantum Volume and CLOPS

how do u compare quantum computers in a hardware-agnostic way? two key metrics.

## 9.1 Quantum Volume

Definition 9.1 (Quantum Volume). quantum volume (QV) is defined as:

$$\mathrm{QV} = 2^{n^*}$$

where $n^*$ is the largest number of qubits $n$ for which the system can successfully execute a random "model circuit" of depth $n$ with heavy output probability $> 2/3$.

the model circuit consists of $n$ layers, each being a random permutation followed by random SU(4) gates on pairs.

Intuition. quantum volume measures the effective "square" of computation the machine can do. its $2^n$ where $n$ is the largest width such that a depth-$n$ circuit (on $n$ qubits) still produces meaningful results. higher QV = can run deeper, wider circuits faithfully.

current records (approximate, 2024):

- IBM: $\mathrm{QV} = 2^{15} = 32768$ (achieved on smaller subsets of large processors)
- Quantinuum: $\mathrm{QV} = 2^{20} = 1048576$ (H2 processor, 56 qubits)
- IonQ: $\mathrm{QV} = 2^{15}$

note that trapped ion systems tend to achieve higher QV despite fewer qubits, bcs their all-to-all connectivity and high gate fidelities mean they can run deeper circuits.

## 9.2 CLOPS — Circuit Layer Operations Per Second

Definition 9.2 (CLOPS). CLOPS measures the throughput of a quantum system:

$$\mathrm{CLOPS} = \frac{M \cdot K \cdot S \cdot D}{T_{\mathrm{wall}}}$$

where $M$ is the number of templates, $K$ is the number of parameter updates, $S$ is the number of shots, $D$ is the number of QV layers, and $T_{\mathrm{wall}}$ is the total wall clock time including all classical overhead.

Key Point. CLOPS is critical for QML bcs variational algorithms like VQE and VQC require many iterations of the optimize-execute loop. if each iteration takes 10 minutes of wall clock time (including queue wait, compilation, execution, result retrieval), and u need 1000 iterations to converge, thats 7 days of wall time. high CLOPS = faster iteration = practical QML.

IBM has pushed CLOPS from ∼1,000 (2021) to ∼100,000+ (2024) through improvements in:

- Qiskit Runtime: executing the optimization loop on classical computers co-located with the quantum hardware
- dynamic circuits: mid-circuit measurement and classical feedforward
- parallelized transpilation and faster data transfer

# 10 Accessing Hardware: Qiskit Runtime Primitives

the modern way to run circuits on IBM hardware is through Qiskit Runtime primitives. there r two:

## 10.1 Estimator

the Estimator primitive computes expectation values:

$$\langle O \rangle = \mathrm{Tr}[\rho\, O]$$

where $\rho$ is the state prepared by ur circuit and $O$ is an observable (usually a Pauli string or sum of Pauli strings).

for QML, u use Estimator when ur model output is a continuous value (like a regression target or classification score computed as $\langle Z_0 \rangle$).

## 10.2 Sampler

the Sampler primitive returns quasi-probability distributions — the probabilities of each bitstring outcome:

$$p(x) = |\langle x|\psi\rangle|^2 \quad \text{for each bitstring } x$$

for QML, u use Sampler when u need the full output distribution, e.g., for quantum kernel estimation where u need $|\langle \phi(x_i)|\phi(x_j)\rangle|^2$.

## 10.3 Error Mitigation

both primitives support built-in error mitigation:

- twirled readout error extinction (TREX): mitigates measurement errors by randomizing the classical bit flip and correcting statistically
- zero noise extrapolation (ZNE): runs the circuit at multiple noise levels (by inserting identity pairs of gates) and extrapolates to the zero-noise limit
- probabilistic error cancellation (PEC): the most accurate but most expensive method — requires full noise characterization and exponential sampling overhead

Example 10.1. a typical QML workflow on IBM hardware:

1. define ur parameterized circuit $U(\theta)$ in Qiskit
2. choose an observable (e.g., $Z_0$ for binary classification)
3. create an Estimator with resilience level 1 (TREX + ZNE)
4. inside the optimization loop: update $\theta$, call Estimator to get $\langle Z_0 \rangle$, compute loss, update $\theta$ via classical optimizer
5. the Estimator handles transpilation, error mitigation, and shot management automatically

# 11 Why Hardware Matters for QML

lets put it all together and understand why all of this directly constrains ur QML models.

## 11.1 Feature Map Depth Constraints

recall that a quantum feature map encodes classical data $x$ into a quantum state:

$$|\phi(x)\rangle = U_\phi(x)\,|0\rangle^{\otimes n}$$

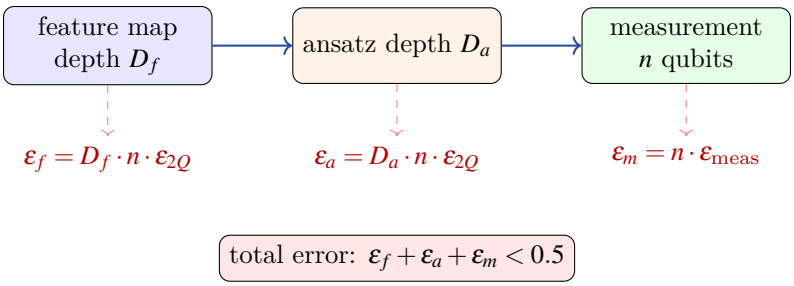
the expressiveness of the feature map depends on its depth — deeper circuits can create more complex features. but hardware limits the usable depth. if ur feature map requires $D$ layers of entangling gates, and the hardware can faithfully execute at most $D_{\max}$ layers, u need $D \leq D_{\max}$.

for current IBM hardware with two-qubit gate error $\sim 0.5\%$:

$$D_{\max} \approx \frac{\ln(2)}{n \cdot \varepsilon_{2Q}} \approx \frac{0.7}{10 \times 0.005} = 14 \text{ layers}$$

(this is very rough — the actual limit depends on the specific circuit structure and error mitigation used.)

## 11.2 Noise Budget Analysis




the total error budget must be divided between:

(1) feature map: needs enough depth to create useful features, but each layer costs noise

(2) ansatz: needs enough parameters for expressiveness, but each entangling layer costs noise

(3) measurement: fixed cost, scales with number of qubits

this creates a fundamental tension in QML: u want deep circuits for expressiveness but shallow circuits for noise. next lecture well see how hardware-efficient ansatze try to get the most expressiveness per unit of noise.

## 11.3 Circuit Depth Limits Per Platform

| Platform | 2Q Error | Max Useful Layers (10Q) | QML Feasibility |
|---|---|---|---|
| IBM Heron | 0.3% | ~20–25 | good for shallow ansatze |
| IBM Eagle | 1% | ~5–10 | marginal |
| Quantinuum H2 | 0.1% | ~50–70 | excellent |
| IonQ Forte | 0.3% | ~20–30 | good |

# 12 Trapped Ions

trapped ion quantum computers use individual atomic ions (typically ytterbium-171 or barium-137) confined in electromagnetic traps. the qubit is encoded in two hyperfine energy levels of the ion.

## 12.1 The Molmer–Sorensen (MS) Gate

the native two-qubit gate for trapped ions is the Molmer–Sorensen (MS) gate:

Definition 12.1 (MS Gate). the MS gate implements:

$$\mathrm{MS}(\theta) = \exp\left(-i\frac{\theta}{4}\hat{\sigma}_x^{(1)} \otimes \hat{\sigma}_x^{(2)}\right)$$

for $\theta = \pi/2$, this creates maximal entanglement:

$$\mathrm{MS}\left(\frac{\pi}{2}\right)|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle - i|11\rangle)$$

the MS gate works by using laser beams to couple the internal qubit states to the shared motional modes (phonons) of the ion chain. the key insight is that the phonon mode acts as a "bus" that mediates entanglement between any pair of ions, regardless of their physical distance in the chain.

## 12.2 All-to-All Connectivity

Key Point. the biggest advantage of trapped ions for QML is all-to-all connectivity. any qubit can directly interact with any other qubit without SWAP routing. this means ur QML circuit runs exactly as designed — no extra SWAP gates, no depth overhead, no additional noise from routing.

for quantum kernel methods, this is particularly valuable. the kernel circuit:

$$|\psi\rangle = U_\phi(x_2)^\dagger U_\phi(x_1)\,|0\rangle^{\otimes n}$$

typically requires entangling gates between many qubit pairs. on trapped ions, the circuit depth for this is exactly the depth of the feature map. on superconducting qubits, it could be 2–3x deeper due to SWAP routing.

## 12.3 Disadvantages

- slow gates: MS gates take $\sim$100 $\mu$s vs. $\sim$50 ns for superconducting. this limits throughput.
- scalability: current systems have 20–56 qubits. scaling to 100+ requires shuttling ions between trap zones.
- CLOPS: much lower than superconducting systems, making iterative QML optimization slow.

# 13 Photonic Quantum Computing

photonic quantum computers use photons (particles of light) as qubits or, more commonly, as continuous-variable (CV) quantum systems.

### 13.1 Continuous Variable Approach

instead of discrete $|0\rangle/|1\rangle$ qubits, CV quantum computing uses the continuous quadratures of the electromagnetic field:

$$\hat{x} = \frac{1}{\sqrt{2}}(\hat{a} + \hat{a}^\dagger), \qquad \hat{p} = \frac{i}{\sqrt{2}}(\hat{a}^\dagger - \hat{a})$$

the native states r gaussian states (coherent states, squeezed states, thermal states) and the native operations r:

- displacement: $D(\alpha) = \exp(\alpha\hat{a}^\dagger - \alpha^*\hat{a})$
- squeezing: $S(r) = \exp\left(\frac{r}{2}(\hat{a}^2 - \hat{a}^{\dagger 2})\right)$
- beam splitter: $BS(\theta) = \exp\left(\theta(\hat{a}_1^\dagger\hat{a}_2 - \hat{a}_1\hat{a}_2^\dagger)\right)$
- Kerr interaction: $K(\kappa) = \exp(i\kappa\hat{n}^2)$ (non-gaussian, needed for universality)

### 13.2 Gaussian Boson Sampling

xanadu's main demonstration has been gaussian boson sampling (GBS) — sampling from the output distribution of a network of squeezed states interfering through a linear optical network:

$$P(n_1, n_2, \ldots, n_M) = \frac{|\mathrm{Haf}(A_S)|^2}{\prod_i n_i! \sqrt{\det(Q)}}$$

where $\mathrm{Haf}(A_S)$ is the hafnian of a submatrix determined by the output pattern. this is classically hard to simulate (under reasonable complexity assumptions).

### 13.3 Photonic QML

for QML, the CV approach offers a different kind of feature map. instead of encoding data into qubit rotations, u can encode it into displacements and squeezings:

$$|\phi(x)\rangle = D(x_1)S(x_2)\cdots|0\rangle$$

xanadu's PennyLane library supports CV quantum computing alongside qubit-based models.

## 14 Neutral Atoms

neutral atom quantum computers use arrays of individual atoms (typically rubidium or cesium) trapped in optical tweezers — tightly focused laser beams that hold atoms in place.

### 14.1 Rydberg Interaction

the key mechanism for entanglement is the rydberg interaction. when an atom is excited to a rydberg state (very high principal quantum number $n \sim 50$–$100$), it has an enormous electric dipole moment, leading to strong van der waals interactions:

$$V_{\text{vdW}}(r) = \frac{C_6}{r^6}$$

where $C_6$ scales as $n^{11}$. this interaction creates a "rydberg blockade": if one atom in a pair is excited to the rydberg state, the other cannot be, bcs the interaction shifts it out of resonance.

Definition 14.1 (Rydberg Blockade). the blockade radius is:

$$R_b = \left(\frac{C_6}{\hbar\Omega}\right)^{1/6}$$

where $\Omega$ is the rabi frequency of the excitation laser. within $R_b$, at most one atom can be in the rydberg state. this naturally implements a controlled-Z type entangling gate.

### 14.2 Reconfigurable Arrays

the killer feature of neutral atoms is reconfigurability. the optical tweezers can be rearranged in real time, allowing u to change the qubit connectivity between different parts of the circuit. this is huge for QML:

- run the feature map with one connectivity pattern
- rearrange atoms
- run the ansatz with a different connectivity pattern
- no SWAP gates needed, ever

QuEra's Aquila processor has demonstrated 256-atom arrays with reconfigurable geometry.

### 14.3 Native Gates

the native operations r:

- global rotations: all qubits rotate simultaneously ($R_x(\theta)$, $R_y(\theta)$)
- local addressing: individual qubit rotations (emerging capability)
- CZ-type gates via rydberg blockade: automatic for atoms within $R_b$

for QML, the global rotation constraint means u design ansatze differently — instead of individual rotations per qubit, u use global pulses and rely on the interaction pattern for expressiveness. this is actually well-suited to QAOA-style circuits.

## 15 Platform Comparison for QML

lets compare all platforms specifically for QML workloads.

| QML Criterion | Supercond. | Trapped Ions | Photonic | Neutral Atoms |
|---|---|---|---|---|
| kernel circuit depth | moderate | low (all-to-all) | N/A (CV) | low (reconfig.) |
| VQC iteration speed | fast (high CLOPS) | slow | moderate | moderate |
| max useful qubits | 50–127 | 20–56 | 100+ modes | 48–256 |
| SWAP overhead | high | none | N/A | minimal |
| error mitigation | mature (Qiskit) | developing | limited | developing |
| software ecosystem | excellent (Qiskit) | good (tket) | good (PennyLane) | emerging |
| cloud access | easy (IBM Q) | available (Azure) | available (Xanadu) | limited |
| best QML use case | VQC (iterative) | kernels | CV QML | QAOA, optimization |

Key Point. there is no single "best" quantum hardware for QML. the right choice depends on ur specific algorithm:

- iterative VQC training with many optimization steps → superconducting (high CLOPS)
- quantum kernel methods with all-to-all entanglement → trapped ions (no SWAP overhead)
- continuous-variable QML or GBS-based kernels → photonic
- QAOA-style optimization with geometry-dependent interactions → neutral atoms

# 16 Practical Considerations

## 16.1 Choosing Qubits on a Backend

not all qubits on a processor r equally good. typical variation on a 127-qubit Eagle:

- best qubits: 2Q error $\sim 0.5\%$, $T_1 \sim 300\,\mu$s
- worst qubits: 2Q error $\sim 3\%$, $T_1 \sim 30\,\mu$s
- thats a 6x variation in quality across the same chip

Qiskit's transpiler can automatically map ur circuit to the best available qubits using the `optimization_level=3` setting.

## 16.2 Shot Budget

each circuit execution (shot) gives one sample from the output distribution. to estimate an expectation value $\langle O \rangle$ to precision $\varepsilon$, u need:

$$N_{\text{shots}} = O\left(\frac{\text{Var}(O)}{\varepsilon^2}\right)$$

for a Pauli observable $O$ with eigenvalues $\pm 1$, the variance is at most 1, so:

$$N_{\text{shots}} \geq \frac{1}{\varepsilon^2}$$

for $\varepsilon = 0.01$ (1% precision), u need $\sim$10,000 shots. at 1000 CLOPS, this takes about 10 seconds. reasonable for one circuit, but a VQC with 100 parameters might need 100 gradient evaluations per step $\times$ 200 steps = 20,000 circuit evaluations. thats $\sim$56 hours of continuous execution.

### 16.3 Error Mitigation Overhead

error mitigation helps but costs additional shots:

- TREX: minimal overhead (∼2x shots)
- ZNE: moderate overhead (∼3–5x shots, running at multiple noise levels)
- PEC: exponential overhead ($\sim e^{2\gamma}$ where $\gamma$ is the total noise strength)

for a practical QML experiment, TREX + ZNE is usually the sweet spot — enough mitigation to get useful results without blowing up the shot budget.

## 17 Summary and Outlook

key takeaways from this lecture:

1. quantum hardware is noisy and limited — this is the dominant constraint on QML today
2. superconducting qubits (IBM) r the most accessible platform with the best software ecosystem
3. gate fidelity (especially 2Q gates) determines ur maximum circuit depth, which is typically 5–25 useful layers on current hardware
4. qubit connectivity determines SWAP overhead — heavy-hex topology means many extra gates for non-local interactions
5. coherence times ($T_1$, $T_2$) set a theoretical upper bound on circuit depth, but in practice gate errors r the bottleneck
6. quantum volume and CLOPS r the key metrics for comparing hardware for QML workloads
7. different platforms have different strengths: superconducting for speed, trapped ions for connectivity, photonic for CV, neutral atoms for reconfigurability
8. always check calibration data and choose qubits carefully when running on real hardware

next lecture well tackle the natural follow-up question: given all these hardware constraints, how do we design ansatze that actually work? thats the topic of hardware-efficient ansatze for QML.